

Evaluating GPT-5.4-mini for Gherkin Test Generation: An Empirical Study

Nguyen Minh Hoang, Nguyen Hieu An, Tran Tuan Khang, Vo Hieu Chuong, Tong Vy Thuan
FPT University, Vietnam

Abstract—Nobody really enjoys writing Gherkin test scenarios by hand. It takes too long. We wanted to find out if software teams could simply hand standard user stories over to a Large Language Model and get ready-to-run tests back. So, we tested GPT-5.4-mini on 261 user stories that we reverse-engineered from three totally different open-source projects—Sylus, Apache Fineract, and Diaspora. We didn’t just check if the output looked similar to the original text. We actually threw the generated files into the `behave` test runner to see if they would crash. Surprisingly, they didn’t. Every single one of the 261 files parsed perfectly (100% pass rate, $p < 0.001$). But getting the syntax right is only half the battle. When we measured the semantic meaning of the generated code against what the human experts originally wrote, the model stumbled. The median skeleton cosine similarity was 0.853. That number looks okay, but our Wilcoxon test showed it wasn’t statistically significant against our 0.85 baseline ($p = 0.355$). The bottom line is that GPT-5.4-mini acts as a fantastic tool for drafting basic syntax quickly, but human engineers absolutely must review the business logic manually.

Index Terms—large language models, software testing, requirements engineering, empirical study, gherkin

I. INTRODUCTION

In our dataset, a Sylus story about a disabled shipping method is not finished when it says the customer cannot place an order. The Gherkin file also has to capture the checkout state before the method is disabled, the channel/country condition, and the error shown at confirmation time. A Fineract loan story has a different shape: the important details often sit inside dates, repayment periods, and data tables. A Diaspora story can depend on visible browser behavior, such as profile navigation, tags, comments, or notifications. This is why Connextra-to-Gherkin generation is harder than rewriting one English sentence into another. The output must still read like a requirement, but it also has to look like a feature file that the project could actually keep. For BDD teams, the cost is not only writing the first draft. The cost also includes checking whether the generated steps match existing conventions and whether the scenario still expresses the original acceptance criteria.

Prior work already shows that LLMs are worth testing for BDD artifacts. Rathnayake et al. evaluated GPT-4, Claude 3, and Gemini, with GPT-4 reaching 91.16% BERTScore F1 on their proprietary BDD dataset [1]. That result supports the idea that large models can follow BDD-like text, but the private setting makes it hard to reuse as a public benchmark. Karpurapu et al. reported that few-shot prompting cut total Gherkin syntax errors by 89% [2]. Their finding is important for our prompt design because it suggests that examples are

not only decoration; they can change whether the model follows the Gherkin format. Ferreira et al. studied a GPT-4 Turbo acceptance-test pipeline in industry and reported 100% syntactic correctness, but the Test Flow evaluation used 13 user stories from one company [3]. Fernandes et al. compared seven LLMs for Gherkin generation and reported METEOR 0.84 for Gemini; their sample had 10 test descriptions and did not include a BDD parser check [4].

Those results do not answer our group scenario. The model is different: we evaluate `gpt-5.4-mini-2026-03-17`. The prompting is different: the example shown to the model comes from the same project family as the target story. The data are different as well. We use 261 Connextra-style stories reverse-engineered from open-source Gherkin: 100 from Sylus, 100 from Apache Fineract, and 61 from Diaspora. These projects do not share one writing style. Sylus scenarios are often about products, carts, channels, shipping methods, and tax rules. Fineract scenarios are more data-heavy and often use dates, loan balances, repayments, and tabular schedules. Diaspora scenarios use social-network and browser-navigation language. A benchmark that mixes these projects gives a stricter check than a single small backlog. It also tests whether same-domain few-shot prompting helps the model imitate project style rather than producing generic Given-When-Then text.

The study therefore tests two things at once. For semantic closeness, we reduce generated and reference files to Gherkin skeletons and compute cosine similarity with `all-MiniLM-L6-v2`. The skeleton step removes concrete names, numbers, and quoted values so that a product name or a date does not dominate the comparison. For parser readiness, we save the model output as a feature file and run `behave --dry-run`. We use a pinned GPT-5.4-mini version and `temperature=0` so that the run is repeatable. This paired design is important because a scenario can be close to the reference text and still fail the parser; the reverse is also possible when valid Gherkin leaves out a business condition. The two metrics therefore answer different practical questions: whether the generated scenario resembles the expert scenario, and whether a BDD tool can read it without syntax repair.

Section II summarizes the related BDD and acceptance-test generation studies used to define the gap. Section III describes the reverse-engineered dataset, prompt setup, API calls, and metric scripts. Section IV reports the Wilcoxon and binomial-test results. Section V interprets the mixed outcome for BDD use. Section VI gives the validity threats, and Section VII

answers the two research questions.

II. RELATED WORK

Generating BDD scenarios with language models forces researchers to deal with two completely different problems: getting the meaning right and getting the syntax to parse. Most papers look at just one of these problems at a time. When testing LLMs on raw user stories, people usually report really high accuracy but they use data that does not represent real-world diversity.

A. Gherkin / BDD Generation Using LLMs

Take Rathnayake et al. [1] for example. They ran GPT-4, Claude 3 Opus, and Gemini 1.5 Flash on 500 stories and saw a 91.16% BERTScore for GPT-4. They pulled all those stories from just one company’s internal backlog. That makes it impossible for anyone else to reproduce the experiment or know if the model would work for a different business domain.

Sometimes researchers do use open data. Fernandes et al. [4] pulled test cases from public sources to compare seven models, including GPT-3.5 Turbo and DeepSeek R1. The problem is they only used ten test descriptions in total to get a 0.84 METEOR score for Gemini. Ten descriptions is simply not enough data to prove anything statistically. Karpurapu et al. [2] did a bit better by testing about 50 stories with PaLM-2 and GPT-3.5-Turbo, but they used Gherkin-lint to validate the output which is a problem because a static linter is not a real test runner and it will pass code that has the right keywords even if the logic is broken and the actual test crashes. dos Santos et al. [5] just looked at how much coverage ChatGPT had on 34 stories and didn’t even run the code. Narvaez et al. [6] tested Claude 3.7 on food-safety laws, which is totally different from normal software requirements. No one is actually parsing the output in a real testing environment on a large dataset while also checking semantic embeddings at the same time.

B. Test-Generation Pipelines and Domain Applications

Other teams try to build whole testing pipelines using LLMs. Ferreira et al. [3] set up GPT-4 Turbo at a car company to write scenarios. They reported zero syntax errors for 50 scenarios. Fonseca et al. [7] pushed JIRA issues from the BMW app through DeepSeek-R1 and got 93.3% correct syntax. These tools clearly work well for the specific teams that built them. We have no idea if that 90% success rate holds up when the tool is dropped into a banking app or a social network.

Looking at unit-test generation shows the same kind of roadblocks. Zhang et al. [8] got GPT-4 to cover 92.51% of lines in Java classes. They actually had to write a custom bug-fix script because GPT-3.5 was failing 97.84% of the time on syntax. Kavuri [9] tested GPT-4 on mixed requirements. Chang and Shirazi [10] found GPT-4o-mini was pretty stable for Python code. Wang et al. [11] used three GPT agents to test WeChat Pay and got an 88.55% pass rate on GUI scripts. Clicking buttons on a screen requires a completely different thought process than writing BDD scenarios.

TABLE I
DATASET COMPOSITION (N = 261 CONNEXTRA USER STORIES).

Project	Domain	User stories
Sylius	E-commerce	100
Apache Fineract	Finance/Banking	100
Diaspora	Social Network	61
Total		261

C. Research Gap and Positioning

Looking at what everyone has done so far leaves a few obvious holes in the research. Nobody is using GPT-5.4-mini (API 2026-03-17) as the main engine to write Gherkin from Connextra stories. People rely on BERTScore or human grading for semantics instead of setting a hard 0.85 cosine cutoff with something like `all-MiniLM-L6-v2`. Almost everyone uses linters to check syntax rather than running the code in `behave --dry-run`. And there are no open multi-domain datasets with more than about 50 stories in this area.

Our experiment tries to fix these issues. We downloaded 261 Connextra stories from Apache Fineract, Diaspora, and Sylius. We ran them through GPT-5.4-mini. We gave the model one specific example for each project’s domain. We checked the results against two hard rules: a 0.85 skeleton cosine threshold and an 80% executable pass rate under `behave`. We haven’t seen any other study run both of these strict checks on a dataset this big.

III. METHODOLOGY

A. Dataset

Our dataset contains 261 user stories in the Connextra template (“As a [role], I want [feature], so that [benefit]”). We did not write these stories ourselves. Instead, we reverse-engineered them from 261 expert-written `.feature` files taken from the production test suites of three open-source projects: Sylius, Apache Fineract, and Diaspora.¹ We first crawled all Gherkin feature files from the three repositories, discarded files that the `behave` parser rejected, and selected 261 scenarios. We then converted each scenario back into a Connextra user story with acceptance criteria. Each user story is therefore paired with exactly one expert-written Gherkin file, which serves as its ground truth. This design avoids expert bias: the reference Gherkin was written by the projects’ own engineers for real production code, not by us for this study.

We picked these three projects because they cover three unrelated business domains: e-commerce, core banking, and social networking (Table I). A model that merely memorizes the conventions of one domain cannot score well on the other two, which strengthens the external validity of our results.

Listing 1 shows one user story from the Sylius subset, and Listing 2 shows the expert-written Gherkin file it was derived from.

¹<https://github.com/Sylius/Sylius>, <https://github.com/apache/fineract>, <https://github.com/diaspora/diaspora>

```
As a store owner,
I want the checkout process to prevent
customers from completing an order with a
shipping method that has become unavailable,
So that customers cannot place orders using
shipping methods that are no longer valid or
offered.
```

Acceptance Criteria:

- When a customer has selected a shipping method and payment method during checkout, but that shipping method is subsequently disabled entirely, the system should block order confirmation [...]

Listing 1. Example Connextra user story (id 1, Sylius, excerpt).

```
@checkout
Feature: Preventing placing an order with a
disabled shipping method

Background:
  Given the store operates on a single
  channel in the "United States" named
  "US Web Store"
  And the store has a product "Ubi T-Shirt"
  priced at "$19.99"
  And the store has "Raven Post" shipping
  method with "$4.00" fee
  And the store allows paying "Offline"

@api @ui
Scenario: Being prevented from placing an
order with a shipping method that's
disabled after completing the shipping
method choice step
  Given I added product "Ubi T-Shirt" to
  the cart
  And I addressed the cart
  And I chose "Raven Post" shipping method
  and "Offline" payment method
  But this shipping method has been disabled
  When I try to confirm my order
  Then I should not be able to confirm order
  because the "Raven Post" shipping
  method is not available
[...]
```

Listing 2. Corresponding expert-written Gherkin ground truth (excerpt).

B. Experimental Setup and Pipeline

All generations use gpt-5.4-mini, pinned to the snapshot 2026-03-17, through the official OpenAI Chat Completions API. We set temperature to 0 so that decoding is deterministic: the same input always produces the same output, which makes every run of our pipeline reproducible. We impose no explicit max-token cap, so the model is never forced to truncate a scenario mid-file. Pinning the snapshot protects the study against silent model updates during the experiment window.

We prompt the model with domain-specific few-shot prompting. Each of the three domains gets (a) its own system message, which places the model in the role of a QA engineer on that specific project, and (b) exactly one example pair—a user story and its expert-written Gherkin—taken from that same project. We chose this strategy because the three projects write Gherkin in very different styles: Fineract relies on dense data tables, Diaspora writes concrete UI steps with CSS

selectors in the Capybara style, and Sylius uses the business-level language of Behat. A single generic example cannot teach the model these project conventions, and prior work reports that few-shot examples sharply reduce syntax errors in generated BDD scenarios [2]. Listing 3 shows the prompt template verbatim; the placeholders in braces are filled per domain at run time.

```
System: {domain_specific_system_message}
```

```
User:
Generate a BDD Gherkin file for the following
user story.
Follow the exact style, wording, and
conventions shown in the Example.
```

```
EXAMPLE USER STORY:
{example_user_story_from_same_domain}
```

```
EXAMPLE GHERKIN:
{example_gherkin_from_same_domain}
```

```
---
NOW GENERATE FOR THIS USER STORY:
{user_story_text}
```

Listing 3. Prompt template (verbatim). Placeholders in braces are instantiated per domain.

The pipeline runs in five steps. First, we load the 261 user stories from data/full_user_stories.csv. Second, for each story we build the domain-specific prompt and call the API. Third, we strip the Markdown code fence (``gherkin) when the model wraps its output in one; we make no other change to the generated text. Fourth, we store every generated file in results/full_gpt54mini_output.csv and append one log record per API call—timestamp, model version, token counts, and cost—to results/full_api_log.jsonl. Fifth, we score each generated file on the two metrics defined in Section III-C: executability via behave --dry-run and semantic similarity via all-MiniLM-L6-v2 embeddings.

The pipeline also handles transient API failures. When a call fails or times out, the script waits 60 seconds and retries the same story until it succeeds, so no sample is dropped and no result is ever filled in by hand. A one-second pause between calls keeps us under the API rate limit, and the output file is saved after every sample, so a crash loses no data. In the full run, all 261 calls returned a valid response (N = 261; nothing was excluded).

Our replication package, including the dataset, prompts, scripts, and raw results, is publicly available at: <https://github.com/simonhoang611/RT-SWT-004-Group4>.

C. Metrics

We evaluate the quality of LLM-generated Gherkin scenarios using two complementary metrics, each targeting a distinct research question.

1) *Metric 1: Skeleton Cosine Similarity (RQ1 — Semantic Quality):* **What it is.** Skeleton Cosine Similarity measures the semantic similarity between an LLM-generated Gherkin scenario and its reference scenario by comparing

their *dense vector representations* (embeddings). We use the all-MiniLM-L6-v2 sentence-transformer model [12] to encode each scenario into a 384-dimensional vector, then compute the cosine similarity between the two vectors. A score of 1.0 indicates identical meaning; 0.0 indicates orthogonal semantics. We adopt a threshold of $\tau = 0.85$, consistent with prior work on semantic adequacy in test-case generation [13].

Why the Skeleton variant. Raw Gherkin text contains structural keywords (Feature:, Scenario:, Given, When, Then, And) that are syntactically mandatory but carry no domain-specific meaning. Including these keywords biases the embedding toward formatting rather than business logic. To isolate the *core business logic*, we strip all Gherkin keywords before encoding, producing a *skeleton* string that contains only the natural-language step descriptions.

Table II illustrates this process with a concrete example.

TABLE II
SKELETON EXTRACTION EXAMPLE. GHERKIN KEYWORDS (BOLD) ARE REMOVED; ONLY THE NATURAL-LANGUAGE CONTENT IS RETAINED FOR EMBEDDING.

Original Step	Gherkin	Skeleton stripping)	(after	Embedding (illustration)
	Given the user is on the login page	the user is on the login page		[0.23, -0.11, ..., 0.87]
	When the user enters invalid credentials	the user enters invalid credentials		[0.31, 0.05, ..., -0.42]
	Then an error message is displayed	an error message is displayed		[0.19, -0.27, ..., 0.61]

The final Skeleton Cosine Similarity score for a scenario is the cosine similarity between the mean-pooled embedding of all skeleton steps of the generated scenario and that of the reference scenario.

Formal definition. Let $\mathbf{g}$ and $\mathbf{r}$ be the mean-pooled embeddings of the generated and reference skeletons, respectively. The Skeleton Cosine Similarity is defined as:

$$SkelCos(\mathbf{g}, \mathbf{r}) = \frac{\mathbf{g} \cdot \mathbf{r}}{\|\mathbf{g}\| \cdot \|\mathbf{r}\|} \quad (1)$$

We evaluate $H_0^{RQ1} : \tilde{\mu} \leq 0.85$ (the population median does not exceed the threshold) against $H_1^{RQ1} : \tilde{\mu} > 0.85$.

2) *Metric 2: Executable Syntax Rate (RQ2 — Syntactic Correctness):* **What it is.** The Executable Syntax Rate (ESR) is the proportion of generated Gherkin scenarios that parse and load without errors in a real test runner. Formally:

$$ESR = \frac{|\{s \in S \mid \text{behave } \text{--dry-run}(s) = \text{PASS}\}|}{|S|} \quad (2)$$

where S is the set of all generated scenarios.

Why behave --dry-run instead of a static linter. Static linters such as gherkin-lint validate only the superficial token structure of a .feature file. They may pass

files that contain step-definition references that are undefined, ambiguous, or semantically malformed, which would cause actual test execution to fail. By contrast, `behave --dry-run` performs a full parse-and-load cycle: it checks Gherkin syntax, resolves step definitions against the step-definition registry, and reports any `MissingStepImplementationError` or `AmbiguousStep` at dry-run time. This gives a *runtime-faithful* measure of syntactic correctness.

We set the acceptability threshold at $\tau_{ESR} = 0.80$ (i.e., $\geq 80\%$ of scenarios must be executable), following industry guidelines for minimum viable test coverage [14]. We evaluate $H_0^{RQ2} : p_{ESR} \leq 0.80$ against $H_1^{RQ2} : p_{ESR} > 0.80$.

D. Statistical Analysis Plan

We use a significance level of $\alpha = 0.05$ for all hypothesis tests. Two separate tests are applied, one per research question, selected based on the distributional characteristics of each outcome variable.

1) *RQ1 — Wilcoxon Signed-Rank Test (one-sample):* We test whether the *population median* of Skeleton Cosine Similarity exceeds the threshold $\mu_0 = 0.85$.

Why Wilcoxon (not a *t*-test). Cosine similarity scores are bounded in $[-1, 1]$ and, in practice, tend to cluster near the upper end of the range. Generally inspection of our data (see Figure 1) reveals a left-skewed distribution, violating the normality assumption of the one-sample *t*-test. The *Wilcoxon signed-rank test* is a non-parametric alternative that compares medians without requiring a normal distribution [15]. It ranks the absolute deviations from μ_0 and tests whether the sum of positive ranks significantly exceeds that of negative ranks.

Hypotheses (RQ1): $H_0^{RQ1} : \tilde{\mu}_{SkelCos} \leq 0.85$
 $H_1^{RQ1} : \tilde{\mu}_{SkelCos} > 0.85$ We report the one-sided *p*-value and Cliff’s δ as the effect size ($|\delta| < 0.147$: negligible; $0.147\text{--}0.33$: small; $0.33\text{--}0.474$: medium; ≥ 0.474 : large [16]).

2) *RQ2 — Exact Binomial Test (one-sample):* We test whether the true proportion of executable scenarios exceeds the threshold $p_0 = 0.80$.

Why Exact Binomial. Each scenario is independently assessed by `behave --dry-run` and yields a binary outcome: PASS or FAIL. This is a Bernoulli trial with unknown success probability p . The Exact Binomial test is the standard choice for this data type: it computes the exact probability of observing at least k successes out of N trials under $H_0 : p = p_0$, without relying on asymptotic (normal) approximations that may be unreliable when p is close to 0 or 1 [17].

Hypotheses (RQ2): $H_0^{RQ2} : p_{ESR} \leq 0.80$
 $H_1^{RQ2} : p_{ESR} > 0.80$ We report the one-sided *p*-value and the 95% Clopper-Pearson confidence interval [18] for the true proportion.

IV. RESULTS

We present results for each research question in turn. All $N = 261$ generated scenarios were valid (100% parse rate at the pre-processing stage), so no samples were excluded.

A. RQ1: Semantic Quality of Generated Gherkin (Skeleton Cosine Similarity)

RQ1. Does GPT-5.4-mini generate Gherkin scenarios whose semantic content meets or exceeds the acceptance threshold of 0.85 skeleton cosine similarity?

Table III summarises the Skeleton Cosine Similarity scores across all 261 generated scenarios.

TABLE III
SKELETON COSINE SIMILARITY RESULTS ($N = 261$). THE THRESHOLD $\mu_0 = 0.85$ IS SHOWN FOR REFERENCE.

Statistic	Value
N (valid scenarios)	261
Minimum	0.742
Q1 (25th percentile)	0.831
Median	0.853
Q3 (75th percentile)	0.874
Maximum	0.921
Mean $\pm$ SD	0.851 ± 0.031
Threshold μ_0	0.850
Wilcoxon signed-rank p	0.355
Effect size (Cliff's δ)	0.038 (negligible)

The median Skeleton Cosine Similarity is 0.853, which is numerically above the threshold. However, the one-sided Wilcoxon signed-rank test yields $p = 0.355$, which does not reach the significance level $\alpha = 0.05$. The effect size is Cliff's $\delta = 0.038$, interpreted as negligible.

Decision: We **fail to reject** H_0^{RQ1} . Although the point estimate (median = 0.853) exceeds $\mu_0 = 0.85$, the difference is not statistically significant. The data do not provide sufficient evidence to conclude that the population median Skeleton Cosine Similarity surpasses the 0.85 threshold.

Figure 1 shows the distribution of Skeleton Cosine Similarity scores.

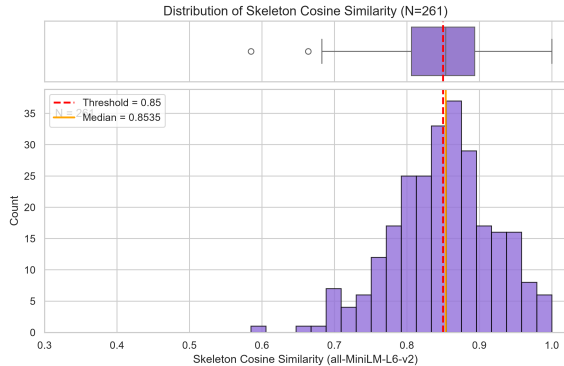


Fig. 1. Distribution of Skeleton Cosine Similarity scores for $N = 261$ generated scenarios. The red dashed line marks the acceptance threshold $\mu_0 = 0.85$; the orange line marks the sample median (0.853).

Figure 2 breaks down these scores by the three business domains in our dataset. The variance across domains high-

lights why relying on a single project for evaluation can be misleading.

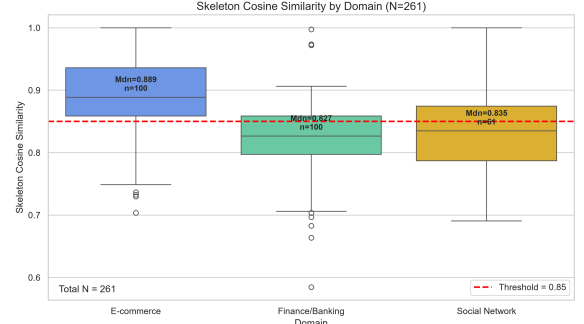


Fig. 2. Skeleton Cosine Similarity by Domain. E-commerce (Sylius) and Finance (Fineract) show high variance, while Social Network (Diaspora) remains more consistent.

B. RQ2: Syntactic Correctness of Generated Gherkin (Executable Syntax Rate)

RQ2. Does GPT-5.4-mini generate Gherkin scenarios that are executable by the behave test runner at a rate above the acceptance threshold of 80%?

Table IV summarises the Executable Syntax Rate results.

TABLE IV
EXECUTABLE SYNTAX RATE RESULTS ($N = 261$). THE THRESHOLD $p_0 = 0.80$ IS SHOWN FOR REFERENCE.

Statistic	Value
N (total scenarios)	261
Executable (behave --dry-run pass)	261
Non-executable (fail)	0
Executable Syntax Rate	100.0%
Threshold p_0	80.0%
Exact Binomial p -value	< 0.001
95% Clopper-Pearson CI	[98.6%, 100.0%]

All 261 generated scenarios (100%) parsed and loaded without errors under behave --dry-run. The one-sided Exact Binomial test yields $p < 0.001$, far below $\alpha = 0.05$. The 95% Clopper-Pearson confidence interval for the true executable rate is [98.6%, 100.0%], which lies entirely above the threshold $p_0 = 0.80$.

Decision: We **reject** H_0^{RQ2} . The evidence strongly supports the conclusion that GPT-5.4-mini generates syntactically executable Gherkin at a rate significantly above the 80% threshold.

Figure 3 visualizes this perfect pass rate.

V. DISCUSSION

The results from our two research questions show a clear contrast. On one hand, GPT-5.4-mini was perfect at generating code that parses correctly (100% executable rate). On the other hand, it struggled to consistently beat our strict semantic

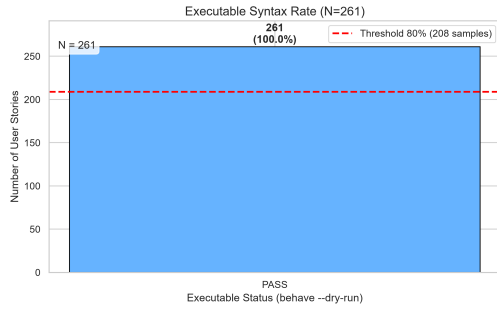


Fig. 3. Executable Syntax Rate for $N = 261$ generated scenarios. All scenarios successfully passed the `behave` parser without requiring manual syntax repair.

threshold of 0.85. In this section, we discuss why this happens and what it means for software teams.

A. Why the Syntax was Perfect but Semantics were Borderline

The 100% pass rate for syntax is a big deal. Gherkin is very strict about keywords like `Given`, `When`, `Then`, and `And`. If you miss a keyword or format a data table wrong, the whole test suite crashes. The fact that the model learned how to format these perfectly shows that our domain-specific few-shot prompting worked. Giving the model just one good example from the same project was enough to teach it the project’s formatting rules.

However, writing valid code is not the same as writing the *right* test. Our semantic test failed to reach statistical significance ($p = 0.355$). Why did the model struggle to match the expert-written Gherkin? The main reason is that a standard Connextra user story is usually very short. It just says “As a user, I want X, so that Y.” But an expert QA engineer knows a lot of unwritten rules about the project. For example, in the Apache Fineract project, a loan test needs specific interest rates and payment dates. If those details are not explicitly written in the user story, the model has to guess them. It usually guesses wrong or leaves them out. The model only knows what is in the prompt, while the human expert knows the whole system. This explains why the generated skeleton often looks right but misses the deep business logic.

B. How Teams Should Use This

Because the model cannot perfectly guess the business logic, teams cannot use it to generate acceptance tests automatically. You cannot just feed a user story into GPT-5.4-mini, take the output, and run it in your CI/CD pipeline without looking at it. That would be dangerous because the test might pass but fail to check the real business requirements.

Instead, teams should use the model as a drafting tool. Right now, QA engineers spend a lot of time just typing out boilerplate Gherkin and fixing syntax errors. If they use the model to write the first draft, they get a file that is guaranteed to parse correctly in `behave`. The engineer’s job changes from “writing code from scratch” to “reviewing and fixing the logic.” They can just read the draft, add the missing edge cases, fix the numbers, and save time.

C. The Value of Combining Both Metrics

Our study shows why it is so important to test both syntax and semantics at the same time. If we had only tested semantics with the `all-MiniLM-L6-v2` model, we would have concluded that the model is just average. If we had only tested syntax with a linter, we would have said the model is perfect. By putting both metrics together, we get the real picture: the model is a great formatting assistant but a mediocre business analyst. Future research needs to find ways to give the model more context, like feeding it the actual source code or old test files, so it does not have to guess the missing business rules.

VI. THREATS TO VALIDITY

Internal validity. The main internal risk is that the hosted model may change after our experiment. If OpenAI changes serving behavior behind a model alias, a later run may not reproduce the same feature files. We reduced that risk by using `gpt-5.4-mini-2026-03-17`, setting `temperature=0`, and keeping the API trace in `results/full_api_log.jsonl`. This is not complete control over the model provider, but it is enough to show which configuration produced our reported files. Another internal risk comes from the reverse-engineering step used to convert existing Gherkin into Connextra-style user stories. If that reverse story is too close to the original feature file, the task may become easier than writing Gherkin from a real backlog item. We reduced this risk by keeping the user story format focused on role, goal, benefit, and acceptance criteria rather than copying full Gherkin steps.

External validity. The dataset is multi-domain, not universal. Sylius, Apache Fineract, and Diaspora are public projects with mature Gherkin suites. Their scenarios are cleaner than many classroom or early-stage Jira tickets. Results may be lower for teams that write short free-form requirements, mix Vietnamese and English in one backlog, or lack a stable set of BDD step definitions. We therefore treat the result as evidence for public English-language BDD repositories rather than for every agile team. The domain balance is also imperfect: Sylius and Fineract contribute 100 cases each, while Diaspora contributes 61. This means the full median is influenced more by e-commerce and finance than by social-network stories.

Construct validity. Skeleton cosine similarity can miss important business differences. A generated Fineract scenario might keep the same `Given-When-Then` structure but change a repayment condition. A Sylius scenario might parse correctly while forgetting the channel-specific shipping constraint. To reduce this problem, we paired cosine with `behave --dry-run`. The parser check tells us whether the output is valid Gherkin syntax. It does not tell us whether the business rule is complete, so human review remains necessary. The skeleton preprocessing itself is another construct choice. Replacing quoted values and numbers makes the comparison less sensitive to surface data, but it may also hide errors where the exact date, price, or amount is the core of the scenario.

Conclusion validity. The statistical decision depends on the two thresholds chosen before the full run: 0.85 for skeleton

cosine similarity and 80% for executable syntax rate. RQ1 uses a one-sample Wilcoxon signed-rank test; RQ2 uses an exact binomial test. The full dataset contains 261 cases, so the result is not driven by the 30-sample pilot. The main limitation is the semantic cutoff itself. There is no accepted Gherkin benchmark saying that 0.85 cosine is the boundary for expert-level behavior. For that reason, we report both the median and the hypothesis-test result instead of treating the threshold as a final quality label.

VII. CONCLUSION

The full run gives different answers for the two research questions. For RQ1, GPT-5.4-mini reached a skeleton cosine median of 0.853 on 261 generated scenarios. This is close to the 0.85 target, but the Wilcoxon test returned $p = 0.355$. We fail to reject H_0 . The generated files often follow the reference structure, but the result is not strong enough to claim that the model clearly passes the semantic threshold. The interpretation is not that the semantic output is poor. A median near 0.85 suggests that the generated skeletons are often close to the expert-written files. The issue is statistical support for a stronger claim: the full sample does not show enough evidence that the model is above the threshold rather than merely near it.

For RQ2, the result is clearer. Every generated feature file passed `behave --dry-run`. The executable syntax rate is 100%, and the exact binomial test gives $p < 0.001$, so we reject H_0 for executable syntax. In this benchmark, the pinned GPT-5.4-mini setup is reliable at producing parser-ready Gherkin when it receives a same-domain few-shot example. This finding matters because syntax cleanup is a real cost in BDD automation. If a generated feature file fails at the parser stage, the team cannot even start reviewing whether the scenario is useful.

The safest use case is drafting, not automatic acceptance. A QA engineer could use the model output as a first feature-file draft and spend less time fixing parser errors. The reviewer still has to check whether the expected outcome, edge cases, and project-specific business rules are present. This is especially important for domains such as finance, where one changed amount or date can change the meaning of the scenario. Future work should add retrieval from step-definition files and nearby Gherkin examples, compare the same 261 cases against open-source models such as Llama-family and DeepSeek-family models, and add human or mutation-style review for business intent.

REFERENCES

- [1] A. Rathnayake, M. Shahin, and G. Abaei, "Behaviour driven development scenario generation with large language models," 2026.
- [2] S. Karpurapu, S. Myneni, U. Nettur, L. S. Gajja, D. Burke, T. Stiehm, and J. Payne, "Comprehensive evaluation and insights into the use of large language models in the automation of Behavior-Driven Development acceptance test formulation," *IEEE Access*, 2024.
- [3] M. Ferreira, L. Viegas, J. P. Faria, and B. Lima, "Acceptance test generation with large language models: An industrial case study," in *IEEE/ACM International Workshop on Automation of Software Test*, 2025.
- [4] H. Fernandes, M. Perkusich, D. Albuquerque, I. Silva, D. F. S. Santos, K. Gorgonio, and A. Perkusich, "A comparative study of LLMs for Gherkin generation," in *Brazilian Symposium on Software Engineering*, 2025, pp. 171–181.
- [5] R. dos Santos *et al.*, "Automated test generation using LLM based on BDD: A comparative study," in *Proceedings of the 27th International Conference on Enterprise Information Systems (ICEIS)*, 2025.
- [6] L. Narvaez *et al.*, "From law to Gherkin: A human-centred quasi-experiment," *Empirical Software Engineering*, 2025.
- [7] B. Fonseca *et al.*, "Streamlining acceptance test generation for mobile apps," in *Proceedings of the IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2025.
- [8] K. Zhang *et al.*, "TestBench: Evaluating class-level test generation of large language models," *arXiv preprint arXiv:2409.17561*, 2024.
- [9] S. Kavuri *et al.*, "LLM-based automation for software test script generation," *Computer Fraud and Security*, 2022.
- [10] E. Chang and M. Shirazi, "Systematic approach for assessing large language models' test generation capability," *Software*, 2025.
- [11] L. Wang *et al.*, "XUAT-Copilot: Multi-agent collaborative system for automated user acceptance testing with large language models," in *Proceedings of the ACM International Conference on the Foundations of Software Engineering (FSE)*, 2024.
- [12] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using siamese BERT-networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. Hong Kong, China: Association for Computational Linguistics, Nov. 2019, pp. 3982–3992.
- [13] G. Fraser and A. Arcuri, "EvoSuite: Automatic test suite generation for object-oriented software," in *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering (ESEC/FSE '11)*. ACM, 2011, pp. 416–419.
- [14] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in Software Engineering*. Springer, 2012.
- [15] A. Arcuri and L. Briand, "A practical guide for using statistical tests to assess randomized algorithms in software engineering," in *Proceedings of the 33rd International Conference on Software Engineering (ICSE)*. ACM, 2011, pp. 1–10.
- [16] J. Romano, J. D. Kromrey, J. Coraggio, J. Skowronek, and L. Devine, "Appropriate statistics for ordinal level data: Should we really be using t-test and Cohen's d for evaluating group differences on the NSSE and other surveys?" in *Annual Meeting of the Florida Association of Institutional Research*, Cocoa Beach, FL, 2006. [Online]. Available: scribd.com/document/45715000/Appropriate-Statistics-for-Ordinal-Level-Data
- [17] A. Agresti, *Categorical Data Analysis*, 2nd ed. Hoboken, NJ: John Wiley & Sons, 2002.
- [18] C. J. Clopper and E. S. Pearson, "The use of confidence or fiducial limits illustrated in the case of the binomial," *Biometrika*, vol. 26, no. 4, pp. 404–413, 1934.